

微分方程数值解大作业 1

庞逸天 523070910095

2026 年 4 月 9 日

1 Task 1

所使用的四个核心算法的 matlab 代码记为 pdf 提供的代码，未做修改，在此不做展示。

在任务 1 中，选择的函数为 $u(x, y) = x(x - 1)y(y - 1)$ ，使用的 matlab 代码为：

```
1 %task1
2 %8-512的网格数按对数均匀分布
3 N_list = unique(round(logspace(log10(8), log10(512), 31)));
4 num = length(N_list);
5
6 %初始化时间矩阵与误差矩阵
7 times = NaN(num, 4);
8 errors = zeros(num, 1);
9
10 fprintf(' N \t\t DST(s) \t BlockLU(s) \t Jacobi(s) \t GS(s)\n');
11 for k = 1:length(N_list)
12     N = N_list(k);
13
14     %DST
15     tic; U_dst = DSTPS(N); times(k,1) = toc;
16
17     %BlockLU
18     tic; U_lu = BlockLU(N); times(k,2) = toc;
19
20     %Jacobi
21     if N <= 128
22         tic; U_jacobi = PSJacobi(N); times(k,3) = toc;
23     else
24         times(k,3) = NaN;
25     end
26
27     %Gauss-Seidel
28     if N <= 128
```

```

29         tic; U_gs = PSGS(N); times(k,4) = toc;
30     else
31         times(k,4) = NaN;
32     end
33
34     %计算精确解与误差
35     h = 1/N;
36     x = h*(1:N-1);
37     [X, Y] = meshgrid(x, x);
38     U_exact = X.*(X-1).*Y.*(Y-1);
39     errors(k) = max(max(abs(U_dst - U_exact)));
40     fprintf(N, times(k,1), times(k,2), times(k,3), times(k,4));
41 end
42
43 %绘制时间图
44 figure('Position', [100, 100, 700, 500]);
45 plot(N_list, times(:,1), '-ro', 'LineWidth', 2, 'MarkerSize', 6); hold on;
46 plot(N_list, times(:,2), '-bs', 'LineWidth', 2, 'MarkerSize', 6);
47 plot(N_list, times(:,3), '-g^', 'LineWidth', 2, 'MarkerSize', 6);
48 plot(N_list, times(:,4), '-md', 'LineWidth', 2, 'MarkerSize', 6);
49
50 set(gca, 'XScale', 'log', 'YScale', 'log');
51 xticks(N_list);
52 xticklabels(string(N_list));
53 grid on;
54
55 xlabel('网格数N(对数均分)', 'FontSize', 12, 'FontWeight', 'bold');
56 ylabel('计算耗时/秒 (对数均分)', 'FontSize', 12, 'FontWeight', 'bold');
57 title('Poisson方程不同求解算法的耗时', 'FontSize', 14);
58 legend('DST', 'LU分解', 'Jacobi 代', 'Gauss-Seidel迭代', ...
59         'Location', 'northwest', 'FontSize', 11);
60
61 %绘制网格数为256数值解与精确解误差图
62 figure;
63 surf(X, Y, abs(U_dst - U_exact));
64 title(['N = ', num2str(N), ' 时 DST的误差']);
65 xlabel('x'); ylabel('y'); zlabel('Error');
66 colormap('jet'); colorbar;

```

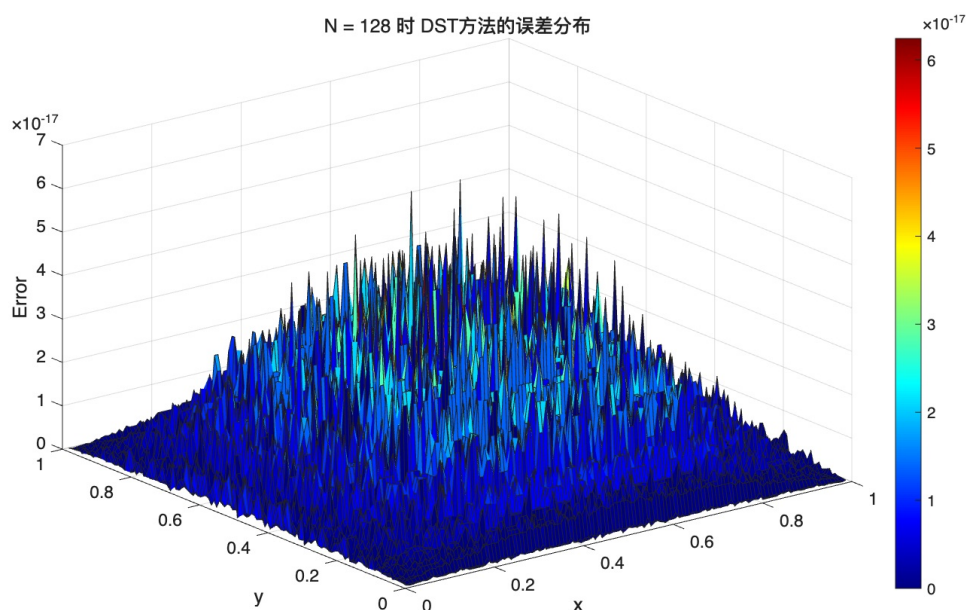
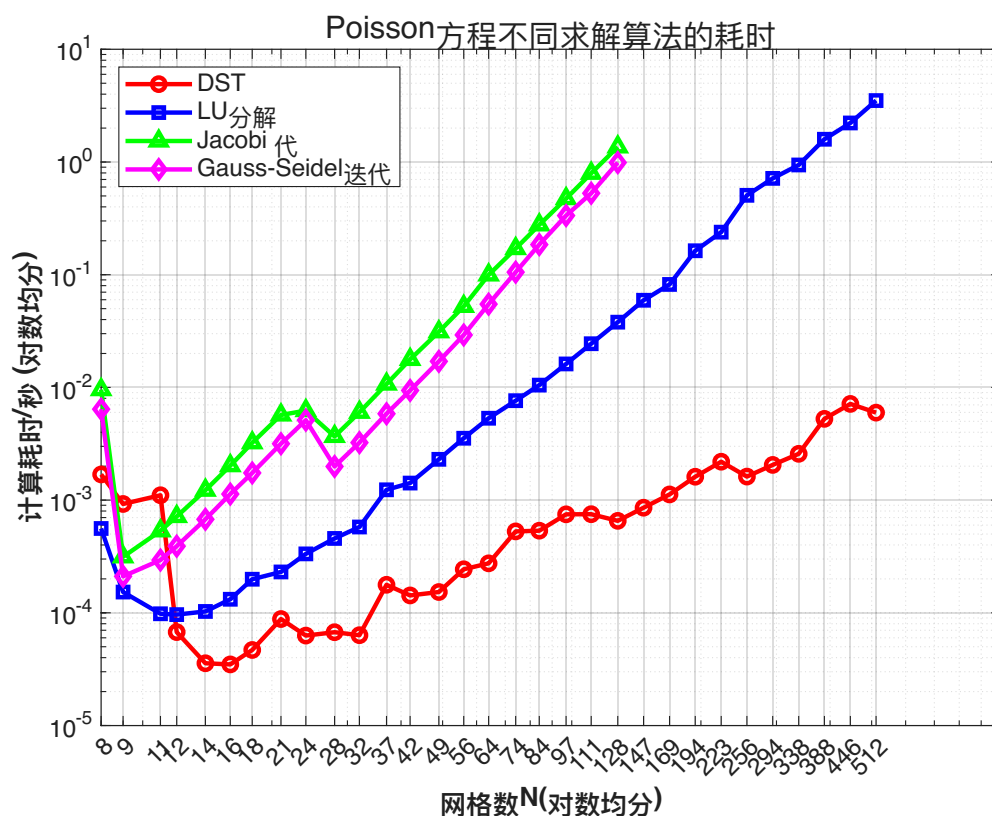
第一部分是对数据的初始化，选择了从 8-512 中按对数均匀分布的 32 个不同的网格数。

第二部分分别使用四种算法在不同网格数下求解 Poisson 方程，并记录每种算法的计算时间。

需要注意的是，由于两种迭代算法在较大网格数下计算时间过长，因此当网格数超过 128 时，直接将其时间记录为 NaN(大网格数的结果会使用图表展示)。

第三部分分别绘制了不同算法的计算时间随网格数变化的图表，以及当网格数为 256 时，DST 算法的数值解与精确解的误差图。

所得到的两张图片为：



除此之外，为了强调迭代算法在大网格数下的计算时间过长，单独绘制了迭代算法的计算时间

图表，结果如下：从图像和表格中可以观察到如下几点：

网格数	DST(s)	LU 分解 (s)	Jacobi 迭代 (s)	Gauss-Seidel 迭代 (s)
8	0.005294	0.001524	0.007225	0.009104
16	0.002533	0.000304	0.002134	0.001513
32	0.000170	0.000820	0.023972	0.007784
64	0.000456	0.005460	0.099198	0.054517
128	0.000890	0.039573	1.368376	0.991336
256	0.001941	0.556518	2.858811	15.815365
512	0.006481	4.081208	331.678322	262.940940

- (1) DST 算法的计算时间远小于其他三种算法
- (2) 两种迭代方法在较大网格数下的计算时间急剧增加
- (3) DST 的时间复杂度相较于其他三者更低

2 Task 2

2.1 理论分析

对于边界条件为非齐次 Dirichlet 条件的二维 Poisson 方程
$$\begin{cases} \Delta u(x, y) = f(x, y), (x, y) \in \Omega \\ u(x, y) = \alpha(x, y), (x, y) \in \partial\Omega \end{cases},$$

如果想要使用 DST 算法对其求解，首先需要将左侧的系数矩阵写成齐次情形时的样子。

考虑横纵坐标的网格数均为 N ，则 $h = k = \frac{1}{N}$ ，那么五点差分格式可以写为：

$$\begin{cases} \frac{u_{i+1,j} + u_{i-1,j} + u_{i,j+1} + u_{i,j-1} - 4u_{i,j}}{h^2} = f_{i,j}, 1 \leq i, j \leq N-1 \\ u_{i,0} = \alpha_{i,0}, u_{i,N} = \alpha_{i,N}, u_{0,j} = \alpha_{0,j}, u_{N,j} = \alpha_{N,j} \end{cases}$$

我们需要对其中包含边界值的项进行修正，使得其满足齐次情形的矩阵。

以 $j = 1$ 为例，有 $u_{i+1,1} + u_{i-1,1} + u_{i,2} + u_{i,0} - 4u_{i,1} = h^2 f_{i,1}$ 。

此处边界值 $u_{i,0} = \alpha(x_i, 0)$ ，希望作调整使得 $u'_{i,0} = 0$ ，则可以将这一项移到右侧

$$u_{i+1,1} + u_{i-1,1} + u_{i,2} + u'_{i,0} - 4u_{i,1} = h^2 f_{i,1} + u_{i,0}$$

其中 $u'_{i,0} = 0$ 。

最后，在等式两边同除 h^2 ，将形式还原

$$\frac{u_{i+1,1} + u_{i-1,1} + u_{i,2} + u'_{i,0} - 4u_{i,1}}{h^2} = f_{i,1} + \frac{u_{i,0}}{h^2}$$

这样，我们可以将非齐次的边界引入到方程的右侧，此时左侧的系数矩阵就满足了齐次情形的形式，可以使用 DST 算法进行求解。

对于其他边界条件的处理方法与上述类似，都是将包含边界值的项移到方程的右侧，将边界值调整为 0。

2.2 算法实现

对给出的 dst 代码稍作修改即可得到满足非齐次边界条件的 DST 算法，具体改动在于需要自行调整方程的右侧矩阵，而不是直接使用原来的 f 矩阵，最终的 matlab 代码如下：

```

1      %task2
2
3      %网格数及步长
4      N = 256;
5      h = 1/N;
6      x = h*(1:N-1);
7      y = h*(1:N-1);
8      [X, Y] = meshgrid(x,y);
9
10     %初始的右侧矩阵
11     F = -4*ones(N-1,N-1);
12
13     %边界处的精确解
14     u_bottom = x.^2;
15     u_top = x.^2+1;
16     u_left = y.^2;
17     u_right = 1+y.^2;
18
19     %为了使左侧矩阵形如齐次情形，将右侧矩阵在临近边界处做调整
20     F(1,:) = F(1,:)+u_bottom / h^2;
21     F(end,:) = F(end,:)+u_top / h^2;
22     F(:,1) = F(:,1)+u_left' / h^2;
23     F(:,end) = F(:,end)+u_right' / h^2;
24
25     %使用快速 DST 求解
26     lambda = 4*sin(pi*x/2).^2;
27     T = dst(dst(F)')';
28     V = zeros(N-1);
29     for i = 1:N-1
30         for j = 1:N-1
31             V(i,j) = T(i,j)/(lambda(i)+lambda(j));
32         end

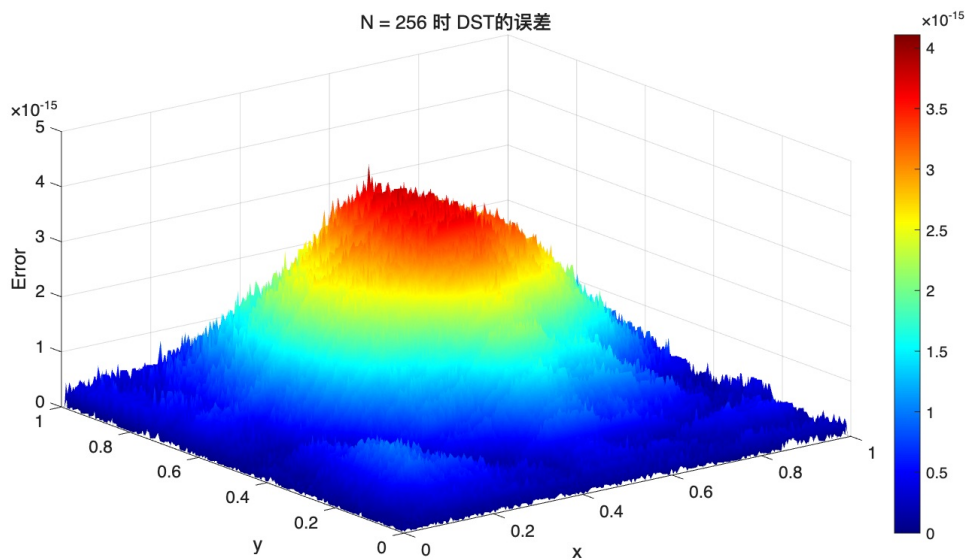
```

```

33 end
34 V = 4*h^4*V;
35 U_num = dst(dst(V))';
36
37 %绘图计算误差
38 U_exact = X.^2+Y.^2;
39
40 figure;
41 surf(X, Y, abs(U_num - U_exact)), shading interp;
42 title(['N = ', num2str(N), ' 时 DST的误差']);
43 xlabel('x'); ylabel('y'); zlabel('Error');
44 colormap('jet'); colorbar;

```

所得到的误差图为：



3 task 3

3.1 理论分析

当边界从 $(0, 1) \times (0, 1)$ 变为 $(0, a) \times (0, b)$ ，且网格数不一定相等后，源代码中 A, B 两个矩阵对应的特征向量相等的特性不再成立，因此我们需要对 A, B 两个矩阵分别计算其特征向量，然后使用 dst 进行求解。

3.1.1 算法实现

```

1 %task 3
2
3 % 设置区域和网格

```

```

4  a = 2;
5  b = 1;
6  Nx = 128;
7  Ny = 256;
8  h = a/Nx;
9  k = b/Ny;
10
11 x = h*(1:Nx-1);
12 y = k*(1:Ny-1);
13 [X, Y]=meshgrid(x,y);
14
15 U_exact = X.*(X-2).*Y.*(Y-1);
16 F = -2*(X.^2-2*X+Y.^2-Y);
17
18 %计算 X 和 Y 方向的特征值
19 lambda_y = (4/k^2)*sin(pi*(1:Ny-1)/(2*Ny)).^2;
20 lambda_x = (4/h^2)*sin(pi*(1:Nx-1)/(2*Nx)).^2;
21
22
23 T = dst(dst(F)')';
24 V = zeros(Ny-1, Nx-1);
25
26 for i = 1:Ny-1
27     for j = 1:Nx-1
28         V(i,j) = T(i,j) / (lambda_y(i) + lambda_x(j));
29     end
30 end
31
32 V = 4/(Nx*Ny)*V;
33 U_num = dst(dst(V)')';
34
35 %计算误差与绘图
36 figure;
37 surf(X, Y, abs(U_num - U_exact)), shading interp;
38 title('一般矩形区域上的误差分布');
39 xlabel('x'); ylabel('y'); zlabel('Error');
40 shading interp; colormap('jet'); colorbar;

```

最终得到的误差图为：

